

Rapport d'Alternance

Ingénieur troisième année

Développement d'un logiciel de documentation des données géographiques



Marcel Assie

Soutenu le 01/09/2026

Organisme d'accueil

ISOGEO
26 Rue du Faubourg Saint-Antoine, 75012 Paris
Equipe Produit

Maître(s) d'apprentissage

Simon SAMPERE

Professeur(s) référent(s)

Laurent BRETON

Rapporteur principal

Période d'alternance : Du 08/09/2025 au 07/09/2026

Nombre de pages : XX

Version du document : 1.0

Remerciements

Je tiens à exprimer mes sincères remerciements à toutes les personnes qui ont contribué, de près ou de loin, au bon déroulement de mon alternance et à la réalisation de ce rapport.

Je remercie tout d'abord Monsieur Mathieu BECKER, directeur d'ISOGEO, de m'avoir accueilli au sein de l'entreprise dans le cadre de ma troisième année de formation d'ingénieur à Géodata Paris. Cette expérience m'a permis d'évoluer dans un environnement professionnel stimulant et de participer directement à l'évolution d'un produit consacré au recensement, à la documentation et à la valorisation des données géographiques.

J'adresse également mes remerciements à Madame Laurence DELAVALD, responsable administrative d'ISOGEO, pour son accueil, sa disponibilité et son accompagnement dans les différentes démarches administratives liées à mon alternance.

Je remercie chaleureusement mon tuteur d'alternance, Simon SAMPERE, pour la confiance qu'il m'a accordée, sa disponibilité et la qualité de son accompagnement tout au long de cette année. Ses conseils techniques et méthodologiques, ainsi que ses retours réguliers, m'ont permis de mieux comprendre les enjeux du produit Scan et d'améliorer ma manière d'analyser, de concevoir et de valider une solution logicielle.

Mes remerciements s'adressent également à l'ensemble des membres de l'équipe Produit d'ISOGEO, notamment Léa, Julie, Léo et Philippe, pour leur accueil, leur disponibilité et les nombreux échanges constructifs qui ont accompagné mon travail. Leur connaissance du produit ainsi que toutes les solutions externes liées au produit ont été essentielles pour comprendre l'existant et proposer des développements cohérents avec l'architecture de Scan.

Je remercie également toutes les personnes ayant participé aux échanges techniques et méthodologiques, aux revues de code et aux différentes phases de validation. Leurs remarques ont contribué à améliorer la robustesse des traitements développés et la qualité générale de la solution.

Je souhaite enfin remercier mon professeur référent, Laurent BRETON, pour son suivi et ses conseils durant cette période d'alternance, ainsi que l'ensemble de l'équipe pédagogique de Géodata Paris pour les connaissances et les méthodes transmises au cours de ma formation d'ingénieur en géomatique.

Enfin, je remercie toutes les personnes qui m'ont soutenu et encouragé tout au long de cette alternance et pendant la rédaction de ce rapport.

Résumé

Le produit Scan d'ISOGEO permet d'explorer différentes sources de données géographiques afin d'identifier leur contenu et d'en extraire les métadonnées. Ces informations alimentent ensuite le processus de catalogage, qui constitue le cœur de l'activité d'ISOGEO en facilitant le recensement, la documentation et la valorisation des données géographiques. Scan prend notamment en charge des bases de données, des géodatabases d'entreprise ainsi que plusieurs formats de fichiers géographiques.

Historiquement, l'ensemble de ces traitements reposait sur des workbenches FME, ce qui entraînait une dépendance externe et des coûts de licence. . ISOGEO a donc engagé leur remplacement progressif par des scripts Python intégrés directement au produit. Ma mission a consisté à poursuivre cette migration, déjà initiée pour certaines sources comme PostGIS et les géodatabases d'entreprise Esri.

Après une phase d'analyse et de documentation de l'existant, j'ai participé au développement de traitements pour plusieurs sources, notamment les données vectorielles et raster, GeoPackage, Oracle et SQL Server. Ces scripts permettent d'extraire les tables, les champs, les projections, les géométries, les emprises spatiales et les autres métadonnées attendues par Scan, ainsi que de générer une signature des données.

La bibliothèque GDAL/OGR a été principalement utilisée, complétée par des bibliothèques adaptées à certains formats spécifiques. Les scripts ont ensuite été organisés dans une architecture modulaire et préparés pour être distribués sous la forme d'un exécutable Windows autonome avec PyInstaller. Cet exécutable est généré dans un environnement Docker afin de rendre le processus reproductible et indépendant des bibliothèques installées sur le poste du développeur. Des tests de régression permettent également de comparer les résultats obtenus avec ceux des anciens traitements FME.

Ces travaux contribuent à réduire la dépendance de Scan à FME, à faciliter la maintenance des traitements et à renforcer leur intégration dans l'environnement logiciel d'ISOGEO.

Mots-clés : géomatique ; Scan ; ISOGEO ; Python ; FME ; GDAL/OGR ; métadonnées ; bases de données spatiales ; Docker ; PyInstaller ; tests de régression.

Abstract

ISOGEO's Scan product explores various geospatial data sources to identify their contents and extract their metadata. This information supports the data cataloguing process, which lies at the core of ISOGEO's activities by facilitating the inventory, documentation and enhancement of geospatial data. Scan supports databases, enterprise geodatabases and several geospatial file formats.

Historically, all of these processes relied on FME workbenches, resulting in an external dependency and additional licensing costs. . ISOGEO therefore began progressively replacing them with Python scripts integrated directly into the product. My work-study assignment consisted in continuing this migration, which had already been initiated for sources such as PostGIS and Esri enterprise geodatabases.

After analysing and documenting the existing developments, I contributed to the implementation of processing components for several sources, including vector and raster data, GeoPackage, Oracle and SQL Server. These scripts extract the tables, fields, coordinate reference systems, geometries, spatial extents and other metadata required by Scan. They also generate data signatures.

GDAL/OGR was the main library used, together with additional libraries adapted to specific formats. The scripts were then organised within a modular architecture and prepared for distribution as a standalone Windows executable using PyInstaller. This executable is built in a Docker environment to ensure a reproducible process independent of the libraries installed on the developer's computer. Regression tests also make it possible to compare the results with those produced by the former FME workflows.

This work contributes to reducing Scan's dependency on FME, facilitating the maintenance of its processing components and strengthening their integration into ISOGEO's software environment.

Keywords : geomatics ; Scan ; ISOGEO ; Python ; FME ; GDAL/OGR ; metadata ; spatial databases ; Docker ; PyInstaller ; regression testing.

Glossaire et liste des sigles

Liste des sigles

Sigle	Signification
API	<i>Application Programming Interface</i> . Interface permettant à plusieurs composants logiciels ou applications de communiquer entre eux.
BDD	Base de données.
CLI	<i>Command-Line Interface</i> . Interface en ligne de commande permettant d'exécuter et de paramétrer un programme depuis un terminal.
CRS	<i>Coordinate Reference System</i> . Système de référence de coordonnées utilisé pour localiser des objets géographiques.
DLL	<i>Dynamic Link Library</i> . Bibliothèque dynamique utilisée par les applications Windows pour partager des fonctions et des dépendances.
EPSG	Référentiel de codes permettant d'identifier les systèmes de coordonnées et les transformations géodésiques.
ETL	<i>Extract, Transform, Load</i> . Processus d'extraction, de transformation et de chargement de données.
FME	<i>Feature Manipulation Engine</i> . Plateforme d'intégration et de transformation de données développée par Safe Software.
GDAL	<i>Geospatial Data Abstraction Library</i> . Bibliothèque libre permettant de lire, écrire et transformer de nombreux formats de données géospatiales.
OGR	Composant de GDAL principalement consacré à la lecture, à l'écriture et au traitement des données vectorielles.
RSE	Responsabilité sociétale des entreprises.
SDE	<i>Spatial Database Engine</i> . Technologie Esri utilisée pour accéder à des données géographiques stockées dans un système de gestion de base de données.
SGBD	Système de gestion de base de données.
SIG	Système d'information géographique.
SQL	<i>Structured Query Language</i> . Langage utilisé pour interroger et manipuler des bases de données relationnelles.
WKT	<i>Well-Known Text</i> . Représentation textuelle normalisée d'une géométrie ou d'un système de coordonnées.

Glossaire

Terme	Définition
ArcPy	Bibliothèque Python fournie par Esri permettant d'automatiser des traitements réalisés avec ArcGIS et d'accéder aux propriétés des données géographiques Esri.
Base de données spatiale	Base de données capable de stocker, d'interroger et de manipuler des objets géographiques tels que des points, des lignes et des polygones.
Connecteur	Composant logiciel chargé d'établir la communication avec une source de données et d'en extraire les informations attendues par l'application.
Donnée raster	Donnée géographique organisée sous la forme d'une grille de cellules ou de pixels, utilisée notamment pour représenter des images aériennes, des modèles numériques ou des cartes continues.
Donnée vectorielle	Donnée géographique représentée par des géométries telles que des points, des lignes ou des polygones, auxquelles peuvent être associés des attributs.
Driver	Composant logiciel permettant à GDAL/OGR de lire ou d'écrire un format de fichier ou une source de données particulière.
Emprise spatiale	Rectangle minimal décrivant les limites géographiques d'un jeu de données. Elle est généralement définie par ses coordonnées minimales et maximales.
Enveloppe convexe	Plus petit polygone convexe contenant l'ensemble des géométries d'un jeu de données.
Géodatabase	Structure de stockage de données géographiques utilisée notamment par les solutions Esri. Elle peut être locale ou hébergée dans un système de gestion de base de données.
Géomatique	Discipline regroupant les méthodes et technologies utilisées pour acquérir, stocker, traiter, analyser et représenter des données localisées.
Métadonnée	Information décrivant une ressource de données, par exemple son nom, sa structure, son format, sa projection, son emprise ou sa date de modification.
Packaging	Processus consistant à regrouper une application et ses dépendances afin de permettre son installation ou son exécution dans un environnement cible.
PostGIS	Extension spatiale du système de gestion de base de données PostgreSQL. Elle permet de stocker et d'interroger des géométries ainsi que d'effectuer des traitements spatiaux.
PyInstaller	Outil permettant de transformer une application Python et ses dépendances en un exécutable pouvant être lancé sans installation préalable de Python.

Terme	Définition
Scan	Produit d'ISOGEO chargé d'explorer des sources de données, d'identifier les ressources qu'elles contiennent et d'extraire les informations techniques nécessaires à leur documentation dans l'écosystème ISOGEO.
Signature	Ensemble d'informations calculées à partir d'une ressource afin de la caractériser, de la comparer ou de détecter une éventuelle évolution de son contenu.
Source de données	Emplacement ou système à partir duquel Scan analyse des ressources. Une source peut notamment correspondre à une base de données, une géodatabase, un dossier ou un fichier géographique.
Test de régression	Test visant à vérifier qu'une modification du logiciel n'altère pas un comportement précédemment validé. Dans ce projet, les résultats Python sont comparés aux résultats produits par les traitements FME de référence.
Workbench FME	Fichier de travail FME décrivant graphiquement une chaîne de lecture, de transformation et d'écriture de données.

Table des matières

Glossaire et liste des sigles	i
Liste des sigles.....	i
Glossaire.....	ii
Liste des figures	vi
Liste des tableaux.....	vii
1 Contexte et objectifs de l’alternance.....	1
1.1 Introduction.....	1
1.2 Présentation du contexte général	2
1.2.1 Présentation de l’organisme d’accueil.....	2
1.2.2 Environnement de travail	4
1.2.3 Approche RSE de l’organisme d’accueil	5
1.3 Présentation du sujet d’alternance	6
1.3.1 Contexte technique du produit Scan.....	6
1.3.2 Migration des traitements FME	8
1.3.3 Problématique et objectifs de la mission	9
2 Prise en main et conception de la solution.....	11
2.1 Découverte de l’environnement Isogeo	11
2.1.1 Formation aux solutions existantes.....	11
2.1.2 Prise en main du produit Scan.....	11
2.2 Analyse et documentation de l’existant	12
2.2.1 Étude du code Python existant	12
2.2.2 Documentation de l’existant	12
2.3 Expérimentations avec GDAL et OGR	13
2.3.1 Prise en main des bibliothèques	13
2.3.2 Réalisation de tests techniques	14
2.4 Conception de la solution.....	14
2.4.1 Définition de l’architecture.....	14
2.4.2 Définition de la méthode de validation	14
2.5 Préparation du packaging.....	15
2.5.1 Objectifs du packaging.....	15
2.5.2 Mise en place d’un environnement Docker	16
3 Développement des scripts Python	18
3.1 Organisation générale des développements.....	18
3.1.1 Une progression par formats de données	18

3.1.2	Périmètre fonctionnel selon les formats	18
3.2	Développements sur les fichiers géographiques	19
3.2.1	Données rasters et vectorielles	19
3.2.2	GeoPackage	20
3.2.3	GeoParquet.....	20
3.2.4	Données CAO.....	21
3.3	Développements sur les bases de données	21
3.3.1	Oracle	22
3.3.2	SQL Server.....	22
3.4	Traitement des nuages de points	23
3.5	Packaging et exécutable au fil des développements	23
3.5.1	Réutilisation de l'environnement Docker	23
3.5.2	Vérification de l'exécutable	23

Table des figures

1.1	Organisation de l'entreprise Isogeo en novembre 2025	3
2.1	Processus de documentation du code source	13
2.2	Passage du code brut au site statique de documentation	13

Liste des tableaux

2.1	Principaux tests prévus pour valider les traitements Python	15
3.1	Progression des développements selon les formats étudiés	19

CHAPITRE 1

Contexte et objectifs de l’alternance

1.1 Introduction

Les données géographiques occupent une place importante dans le fonctionnement de nombreuses organisations. Elles sont utilisées dans des domaines variés tels que l’aménagement du territoire, la gestion des réseaux, l’environnement, les transports ou encore l’urbanisme. Cependant, ces données sont souvent réparties entre plusieurs bases de données, serveurs, répertoires de fichiers et applications métiers. Cette dispersion peut rendre leur identification, leur compréhension et leur réutilisation difficiles.

Isogeo développe des solutions permettant aux organisations de mieux recenser, documenter et valoriser leur patrimoine de données géographiques. Parmi ces solutions, le produit Scan permet d’explorer différentes sources de données, d’identifier les ressources qu’elles contiennent et d’en extraire les informations techniques nécessaires à leur catalogage. Il peut notamment analyser des bases PostgreSQL/PostGIS, Oracle et Microsoft SQL Server, des géodatabases d’entreprise Esri ainsi que plusieurs formats de fichiers géographiques.

Historiquement, l’ensemble des traitements réalisés par Scan reposait sur des workbenches FME. Ces traitements permettaient de parcourir les différentes sources, d’analyser les ressources disponibles et de produire les informations attendues par le produit. Toutefois, cette organisation rendait le fonctionnement de Scan dépendant de l’installation et de la licence d’un logiciel externe.

Dans une démarche d’évolution du produit, Isogeo a donc engagé la migration progressive de l’ensemble de ces traitements FME vers une solution développée en Python. L’objectif est de rendre Scan plus autonome, de faciliter la maintenance de ses traitements et de permettre leur exécution sous la forme d’un programme Windows ne nécessitant pas l’installation préalable de Python ou de FME.

Mon alternance s’inscrit dans ce contexte. Ma mission principale consiste à poursuivre cette migration en analysant les traitements existants, leurs entrées, leurs sorties et les règles métier qu’ils appliquent, puis en développant des scripts Python capables de produire des résultats équivalents. Elle comprend également la prise en charge de différentes sources de données, la gestion des erreurs, la mise en place de tests comparatifs et la préparation d’un exécutable Windows autonome.

La problématique générale de ce travail peut ainsi être formulée de la manière suivante :

Comment remplacer intégralement les traitements FME utilisés par Scan par

une solution Python autonome, tout en garantissant la fiabilité des résultats et la compatibilité avec les différentes sources de données prises en charge ?

Pour répondre à cette problématique, le travail a été organisé autour de l'analyse de l'existant, du développement des nouveaux traitements, de leur intégration dans l'architecture de Scan et de leur validation par comparaison avec les résultats FME de référence.

Ce rapport est structuré en quatre chapitres. Le premier présente le contexte de l'alternance, l'entreprise Isogeo, le produit Scan ainsi que les objectifs de la mission. Le deuxième expose l'analyse de l'existant et les choix techniques retenus pour concevoir la solution. Le troisième décrit les développements réalisés, la préparation de l'exécutable et la démarche de validation. Enfin, le quatrième présente les résultats obtenus, les apports de l'alternance, les limites rencontrées et les perspectives du projet.

1.2 Présentation du contexte général

1.2.1 Présentation de l'organisme d'accueil

Isogeo est une entreprise française spécialisée dans le recensement, la documentation et la valorisation des données géographiques. Elle développe une solution de catalogage destinée aux organisations qui produisent, administrent ou utilisent un volume important de données géographiques.

Son activité répond à une difficulté fréquemment rencontrée au sein des systèmes d'information : les données sont souvent réparties entre plusieurs bases de données, serveurs, répertoires de fichiers et applications métiers. Cette dispersion peut rendre difficile l'identification des ressources disponibles, la compréhension de leur contenu, l'évaluation de leur qualité et leur réutilisation par les différents utilisateurs.

La solution développée par Isogeo permet d'automatiser la constitution et la mise à jour d'un catalogue de données géographiques. Elle aide les organisations à disposer d'un inventaire centralisé de leur patrimoine de données, à enrichir les fiches de métadonnées associées et à faciliter leur recherche, leur consultation et leur diffusion.¹

Les utilisateurs de la solution appartiennent aussi bien au secteur public qu'au secteur privé. Il peut notamment s'agir de collectivités territoriales, de bureaux d'études ou de grandes organisations disposant d'un patrimoine important de données géographiques.

1. [Site officiel d'Isogeo](#) Site officiel d'Isogeo, présentation de la solution de catalogage, consulté en juillet 2026.

Organisation de l'entreprise

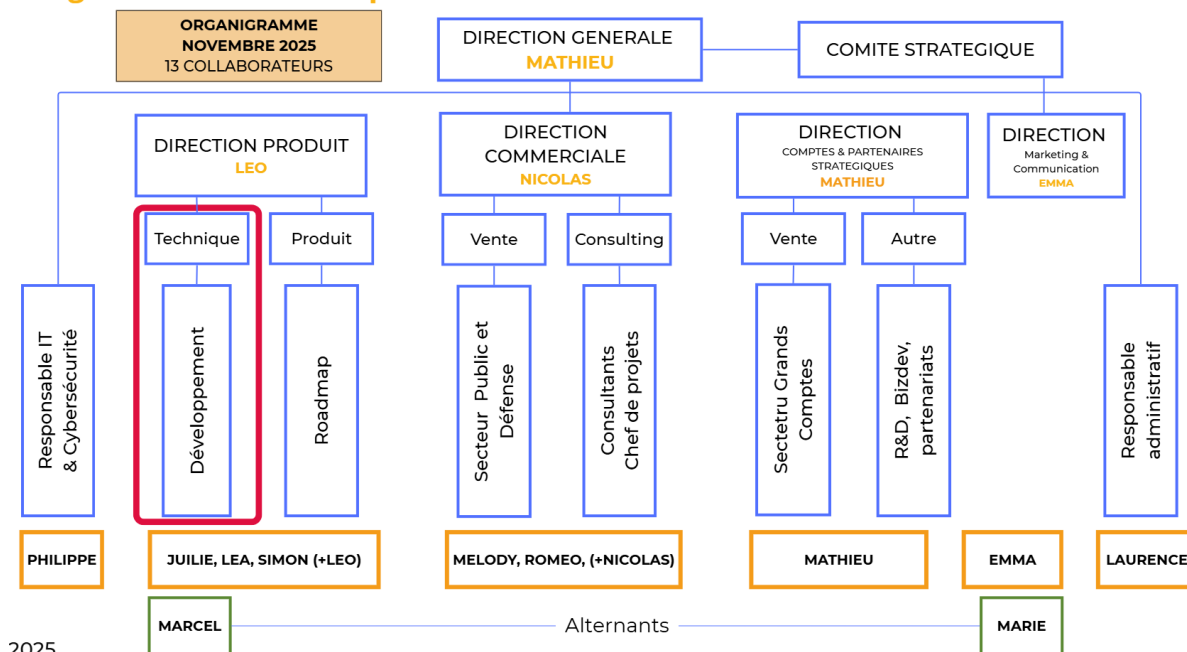


FIGURE 1.1 : Organisation de l'entreprise Isogeo en novembre 2025

Produits et service

L'écosystème Isogeo s'organise autour de plusieurs services complémentaires, chacun associé à un usage précis dans le cycle de vie des catalogues de données. L'application principale, appelée *App*, constitue l'espace de travail des utilisateurs. Elle sert à recenser les jeux de données, à documenter les fiches de métadonnées, à les enrichir et à valoriser les catalogues auprès des publics concernés. Le service *Scan* intervient plus en amont : il permet de repérer automatiquement les données présentes dans les fichiers, les bases de données ou les serveurs cartographiques afin de préparer ou mettre à jour les fiches associées.

La partie *Manage* est destinée à l'administration de la solution. Elle permet de gérer les groupes de travail, les utilisateurs, les applications, les invitations ainsi que les listes de mots-clés et de thématiques utilisées pour structurer les catalogues. D'autres services sont tournés vers la diffusion et l'accès aux données. L'*Extracteur* permet de télécharger des données dans différents formats et projections, tandis qu'*OpenCatalog* sert à publier un catalogue sur le web et à le rendre consultable depuis un site ou un portail. Le *Portail de données* propose une interface de diffusion et de valorisation plus complète, avec des fonctionnalités de recherche, de suivi et d'animation autour des catalogues publiés. Enfin, les plugins pour QGIS et ArcGIS Pro permettent aux utilisateurs de retrouver, consulter et ajouter des données directement depuis leur logiciel SIG habituel, à partir des métadonnées documentées dans Isogeo.²

2. Base de connaissance officielle des produits Isogeo Base de connaissance officielle des produits Isogeo, consultée en juillet 2026.

1.2.2 Environnement de travail

L’alternance s’est déroulée au sein de l’équipe Produit d’Isogeo. Cette équipe participe à la conception, au développement, à la maintenance et à l’amélioration continue des différents composants de la solution.

Mon travail a été réalisé sous la responsabilité de Simon SAMPERE , maître d’apprentissage, et en collaboration avec les membres de l’équipe impliqués dans le développement et la maintenance du produit Scan.

Organisation du travail

Durant les premiers mois, un point de suivi quotidien était organisé avec mon tuteur d’alternance. Il permettait de revenir sur les tâches réalisées la veille, de définir les priorités de la journée et de suivre mon intégration au sein de l’entreprise.

Le travail s’inscrivait aussi dans une organisation agile. Chaque matin, l’équipe participait à un *stand-up* d’environ quinze minutes. Chacun y présentait ce qu’il avait fait, ce qu’il prévoyait de faire et les éventuels points de blocage.

D’autres réunions rythmaient les sprints. Les *sprint reviews* permettaient de présenter les travaux réalisés. Les *sprint retrospectives* servaient à identifier les points d’amélioration dans l’organisation de l’équipe. Les *planning meetings* permettaient enfin de préparer les tâches du sprint suivant.

Le suivi reposait également sur un système de tickets. Chaque ticket décrivait une fonctionnalité, une anomalie ou une amélioration attendue. Cette organisation facilitait la priorisation des travaux et permettait de conserver une trace des décisions prises.

Outils utilisés

La réalisation de la mission a nécessité l’utilisation de plusieurs outils :

- des IDE pour écrire, exécuter et déboguer les scripts Python ;
- Git pour la gestion des versions et le suivi des modifications ;
- FME pour consulter et exécuter les workbenches servant de référence ;
- GDAL et OGR pour traiter certains formats de données géographiques ;
- ArcGIS Pro et ArcPy pour réaliser des tests propres à l’environnement Esri ;
- des bases de données hébergées sur des serveurs pour tester les scripts ;
- Docker et des machines virtuelles pour reproduire les environnements d’exécution et de test ;

Collaboration et contrôle de la qualité

La collaboration au sein de l'équipe reposait sur des échanges techniques, des revues de code et différentes phases de test. Les modifications étaient enregistrées dans le gestionnaire de versions afin de suivre leur évolution, de les comparer et de les intégrer progressivement au projet.

Une attention particulière était portée à la lisibilité du code, à la gestion des erreurs et à la stabilité des échanges entre les différents composants. Les scripts développés devaient en effet s'intégrer dans une chaîne de traitement existante et respecter des formats d'entrée et de sortie déjà utilisés par les autres parties de Scan.

La documentation constituait également un élément important du travail. Elle permettait de présenter le comportement des traitements, leurs dépendances, leurs paramètres et les éventuelles limites identifiées.

1.2.3 Approche RSE de l'organisme d'accueil

La responsabilité sociétale des entreprises, généralement désignée par le sigle RSE, correspond à la prise en compte des enjeux environnementaux, sociaux et économiques dans les activités et les décisions d'une organisation.

La présente partie ne constitue pas un audit de la démarche RSE d'Isogeo. Elle vise à mettre en évidence les liens pouvant être établis entre les activités de l'entreprise, son organisation et certains principes associés à la responsabilité sociétale.

Gouvernance et valorisation des données

La principale contribution de la solution Isogeo concerne la gouvernance et la valorisation du patrimoine de données des organisations. La mise en place d'un catalogue permet de mieux identifier les ressources existantes, leurs producteurs, leurs conditions d'utilisation et leur niveau de documentation.

Cette connaissance favorise la réutilisation des données disponibles et limite le risque de produire plusieurs fois une information déjà existante. Elle contribue également à améliorer la traçabilité des données et à fiabiliser les processus métiers qui en dépendent.

Pour les organismes publics, une meilleure documentation facilite également la diffusion et l'ouverture des données. Elle peut ainsi contribuer à renforcer la transparence de l'action publique et l'accès aux informations produites par les administrations.³

Enjeux environnementaux

Les activités numériques ont un impact environnemental lié notamment à l'utilisation des infrastructures informatiques, au stockage des données et à la consommation de ressources de

3. [Isogeo, exemple de catalogage et de publication de données ouvertes](#) Isogeo, exemple de catalogage et de publication de données ouvertes, consulté en juillet 2026.

calcul. Une gestion plus rigoureuse des données peut contribuer indirectement à limiter certains traitements inutiles ou redondants.

Le fonctionnement de Scan s'inscrit dans cette logique. Une signature est calculée à partir du contenu des données afin de détecter leur modification. Lorsqu'une ressource n'a pas changé depuis l'exécution précédente, il n'est pas nécessaire de réaliser de nouveau l'ensemble de son traitement. Ce mécanisme permet d'optimiser les temps d'exécution et d'éviter certaines opérations inutiles.⁴

La migration de FME vers une solution Python autonome permet également de simplifier les prérequis nécessaires au fonctionnement du produit. Aucun gain environnemental précis ne peut toutefois être affirmé sans une mesure comparative des ressources consommées avant et après la migration.

Transmission des connaissances et qualité du travail

La documentation et le partage des connaissances occupent une place importante dans le fonctionnement de l'équipe. Les revues de code, les échanges techniques et la rédaction de documentation rendent les développements plus compréhensibles et limitent la dépendance à une seule personne.

L'accueil d'étudiants en stage ou en alternance participe également à la transmission des compétences dans les domaines du développement logiciel, de la géomatique et de la gouvernance des données.

Enfin, les solutions développées par Isogeo rendent les données géographiques plus facilement accessibles à leurs utilisateurs. En facilitant leur recherche, leur compréhension et leur réutilisation, elles contribuent à réduire certains obstacles techniques liés à leur exploitation.

1.3 Présentation du sujet d'alternance

1.3.1 Contexte technique du produit Scan

Scan est le composant de la solution Isogeo chargé d'explorer les sources de données configurées par les utilisateurs. Il identifie les ressources présentes, en extrait les informations techniques, puis transmet les résultats à la plateforme Isogeo afin de créer ou de mettre à jour les fiches de métadonnées correspondantes.

Le service installé dans l'environnement du client est appelé *Isogeo Worker*. Il s'exécute sur une machine Windows disposant des droits nécessaires pour accéder aux sources ciblées. Les opérations réalisées par Scan sont des opérations de lecture et n'ont pas pour objectif de modifier les données des clients.⁵

4. [Isogeo, détection de la modification des tables géographiques](#) Isogeo, « Détection de la modification des tables géographiques », septembre 2022.

5. [Documentation officielle de Scan](#) Documentation officielle de Scan, présentation du sous-programme

Le traitement d'une source repose principalement sur trois opérations : le listing, la signature et l'extraction des métadonnées.

Le listing

Le listing consiste à inventorier les ressources accessibles à partir d'un point d'entrée. Selon le type de source, une ressource peut correspondre à une table, une vue, une couche géographique, un fichier ou un jeu de données.

Cette étape permet à Scan de connaître la liste des éléments disponibles avant de déterminer ceux qui doivent être analysés plus précisément.

La signature

La signature correspond à une empreinte calculée à partir du contenu d'une ressource. Elle permet à Scan de reconnaître une donnée déjà rencontrée et de détecter une éventuelle modification entre deux exécutions.

Cette information joue un rôle important dans la mise à jour du catalogue. Si une donnée n'a pas changé, certains traitements peuvent être évités. Si sa signature évolue, Scan peut déclencher une nouvelle extraction de ses métadonnées.

La documentation

L'étape de documentation des données, également appelée *lookup* dans la documentation technique de Scan, produit les informations décrivant la ressource analysée. Elle peut par exemple récupérer son nom, son emplacement, son format, son système de coordonnées ou encore le nombre d'entités qu'elle contient.

Selon le type de donnée, Scan peut aussi extraire des informations plus spécifiques, comme le type de géométrie, l'emprise spatiale ou certaines propriétés propres aux données raster. Ces éléments servent ensuite à compléter la fiche de métadonnées associée.

Les résultats doivent être structurés de manière à pouvoir être interprétés par les autres composants de Scan. La nouvelle implémentation Python doit donc respecter le contrat déjà utilisé par le produit, notamment la structure des résultats, les paramètres d'exécution et les codes d'erreur.

Diversité des sources prises en charge

Scan doit fonctionner dans des environnements techniques variés. Les données peuvent être stockées dans des bases de données, des géodatabases Esri, des fichiers vectoriels ou raster, ou encore être accessibles depuis certains services géographiques.

Cette diversité rend le produit plus complexe à maintenir. Chaque source a ses propres règles de connexion, ses formats et ses comportements. Une même opération peut donc nécessiter un traitement différent selon la technologie utilisée.

d'extraction de métadonnées, consultée en juillet 2026.

1.3.2 Migration des traitements FME

FME est une plateforme spécialisée dans l'intégration et la transformation de données. Elle propose de nombreux lecteurs et composants adaptés aux données géographiques. Pour cette raison, l'ensemble des traitements historiques de Scan reposait sur des workbenches FME.

Le choix d'engager une migration ne remet pas en cause les capacités fonctionnelles de FME. Il répond principalement à un besoin d'autonomie et d'évolution du produit.

Dépendance à un logiciel externe

L'exécution des workbenches nécessite une installation compatible de FME ainsi qu'une licence disponible. Le fonctionnement de Scan dépend donc d'un logiciel tiers qui doit être installé, configuré et maintenu dans l'environnement du client.

Cette dépendance augmente le nombre de prérequis nécessaires au déploiement du produit et entraîne des coûts de licence. Elle peut également compliquer les opérations de support lorsqu'un problème dépend de la version de FME, de l'activation de la licence ou d'un composant particulier de l'installation.⁶

Maintenance des traitements

Les workbenches FME offrent une représentation graphique des traitements, mais leur maintenance peut devenir complexe lorsque le nombre de sources et de cas particuliers augmente.

Une implémentation Python permet de regrouper les comportements communs dans des classes et des fonctions réutilisables. Elle facilite également la gestion des versions, les revues de code, la mise en place de tests automatisés et l'intégration d'outils de contrôle de la qualité.

Intégration dans l'architecture de Scan

Le remplacement des workbenches par des composants Python permet de rapprocher les traitements de données du reste du code de Scan.

Les scripts doivent également être distribués sous la forme d'un exécutable Windows autonome. La génération de cet exécutable repose sur PyInstaller. Elle est réalisée dans un environnement Docker afin de disposer d'un processus reproductible et indépendant des bibliothèques installées sur le poste du développeur.

Continuité fonctionnelle

Dans un premier temps, l'objectif de la migration est de conserver une isofonctionnalité avec les traitements FME existants. Les nouveaux traitements Python doivent donc produire des résultats compatibles avec ceux attendus par Scan et par les autres composants de la solution.

6. [Documentation officielle de support de Scan](#) Documentation officielle de support de Scan, consultée en juillet 2026.

Au cours du travail, certains ajustements ont toutefois pu être réalisés. Ils permettaient de corriger des écarts, de simplifier certains traitements ou d'adapter le fonctionnement aux contraintes propres à chaque source de données.

La migration est réalisée progressivement, source par source. Avant le début de mon alternance, ce travail avait déjà été engagé pour certaines sources, notamment PostGIS et les géodatabases d'entreprise Esri.

1.3.3 Problématique et objectifs de la mission

La problématique générale de cette alternance peut être formulée de la manière suivante :

Comment remplacer intégralement les traitements FME utilisés par Scan par une solution Python autonome, tout en garantissant la fiabilité des résultats et la compatibilité avec les différentes sources de données prises en charge ?

Ma mission s'inscrit dans la poursuite de cette migration. Elle ne consistait pas à développer un nouveau produit indépendant, mais à faire évoluer un composant existant tout en maintenant sa compatibilité avec le reste de l'écosystème Isogeo.

Objectif principal

L'objectif principal était de concevoir, développer et valider des traitements Python capables de remplacer les traitements FME utilisés par Scan pour inventorier, signer et documenter différentes sources de données.

Les traitements développés devaient pouvoir être intégrés dans l'architecture existante et être exécutés de manière autonome dans un environnement Windows.

Objectifs spécifiques

Pour atteindre cet objectif principal, plusieurs objectifs spécifiques ont été définis :

- comprendre et documenter l'existant, comprendre l'architecture de Scan et les traitements FME servant de référence ;
- préparer un exécutable Windows autonome, généré dans un environnement Docker reproductible.
- développer en Python les opérations principales de Scan, notamment le listing, la signature et l'extraction des métadonnées ;
- comparer les résultats obtenus avec ceux de FME et mettre en place des tests de régression ;

Les principales sources étudiées au cours de la mission comprenaient notamment les formats de

données vectorielles et raster, les données CAO (DWG, DXF), GeoPackage, Oracle, Microsoft SQL Server, GeoParquet et les nuages de points.

Périmètre et limites

Certaines limitations dépendaient des technologies utilisées, notamment pour les données CAO. Tous les formats n'ont pas pu être pris en charge dans le périmètre de la mission. Les formats .SDF et .DGN ont ainsi été mis de côté.

Pendant la migration, les anciens traitements FME ont été conservés comme référence. Ils permettaient de comparer les résultats, d'identifier les écarts et de vérifier que la nouvelle solution respectait le comportement historique attendu par Scan.

La réussite de la mission reposait donc sur la qualité des développements Python, leur intégration dans le produit et la mise en place d'une méthode de validation fiable. Cette validation était nécessaire pour vérifier l'équivalence entre les nouveaux traitements et les traitements FME de référence.

CHAPITRE 2

Prise en main et conception de la solution

2.1 Découverte de l'environnement Isogeo

2.1.1 Formation aux solutions existantes

Les premières semaines ont été consacrées à la découverte des solutions développées par Isogeo. Cette phase correspondait à la période d'intégration. Elle devait me permettre de comprendre l'entreprise, son organisation et le rôle de Scan dans l'ensemble de l'écosystème.

Au cours de cette période, j'ai regardé plusieurs webinaires de présentation de la solution. J'ai également lu des documents liés à l'entreprise, aux produits Isogeo et aux pratiques de développement mises en place au sein de l'équipe Produit.

Cette étape était importante pour mieux situer la mission dans le fonctionnement réel du produit.

Les formations ont porté sur les principaux services utilisés par les clients. *L'App* constitue l'espace de travail principal pour gérer les catalogues et documenter les fiches de métadonnées. *Manage* est destiné à l'administration des groupes, des utilisateurs et des applications associées. *OpenCatalog* et le *Portail de données* servent à publier et à valoriser les catalogues auprès des utilisateurs finaux.

D'autres outils complètent cet ensemble. *L'Extracteur* permet de rendre les données plus facilement téléchargeables. Les plug-ins QGIS et ArcGIS Pro facilitent l'accès aux données depuis les logiciels SIG utilisés au quotidien par les géomaticiens.

Cette découverte m'a permis de comprendre que Scan n'est pas un composant isolé. Il intervient en amont du catalogue. Son rôle est de recenser les ressources disponibles, d'en extraire les informations techniques utiles et de préparer la mise à jour des fiches de métadonnées.

2.1.2 Prise en main du produit Scan

La prise en main de Scan a ensuite permis d'identifier les principales étapes réalisées par le produit. Les traitements reposent sur trois opérations centrales : le listing, la signature et la documentation des données.

Le listing sert à inventorier les ressources accessibles dans une source. La signature permet de détecter si une ressource a changé depuis une exécution précédente. La documentation produit les informations techniques nécessaires à la création ou à la mise à jour d'une fiche de

métadonnées.

Cette phase a aussi permis de comprendre les contraintes du produit. Le logiciel doit fonctionner chez les clients, dans des environnements différents, avec des sources de données variées. Les traitements doivent donc être fiables, prévisibles et compatibles avec le fonctionnement existant.

La prise en main ne s'est pas limitée à une lecture théorique. Elle s'est faite progressivement, en échangeant avec l'équipe, en observant le comportement du produit et en réalisant des premiers tests sur des sources simples.

2.2 Analyse et documentation de l'existant

2.2.1 Étude du code Python existant

Avant mon arrivée, une partie de la migration avait déjà été engagée. Il existait donc du code Python à lire, à comprendre et à replacer dans l'architecture générale de Scan.

Cette étape a consisté à parcourir les modules existants, à identifier les classes principales et à comprendre la manière dont les traitements étaient organisés. L'objectif était de repérer les éléments déjà réutilisables et les choix de conception à conserver pour la suite du développement.

L'analyse du code a aussi permis de mieux comprendre les responsabilités de chaque module. Certains éléments étaient communs à plusieurs sources, comme la gestion des paramètres, la structure des résultats ou le traitement des erreurs. D'autres dépendaient directement du type de source analysée.

Cette lecture progressive a servi de base au travail de développement. Elle a évité de repartir de zéro et a permis de rester cohérent avec les choix déjà faits par l'équipe.

2.2.2 Documentation de l'existant

La documentation a accompagné cette phase d'analyse. Elle avait pour but de conserver les informations utiles à la poursuite de la migration et de rendre plus lisible le fonctionnement du code existant.

Il s'agissait principalement d'une documentation directement liée au code source. Les classes et les fonctions étaient décrites à l'aide de commentaires de documentation, placés sous leur définition. Ces descriptions précisaient le rôle du traitement, les paramètres attendus, les valeurs retournées et les cas particuliers à connaître.

Cette documentation était utile pendant la prise en main, mais aussi pour toute personne devant éventuellement reprendre le code source. Elle facilitait les échanges techniques et permettait de partager plus facilement ce qui avait été compris ou vérifié.

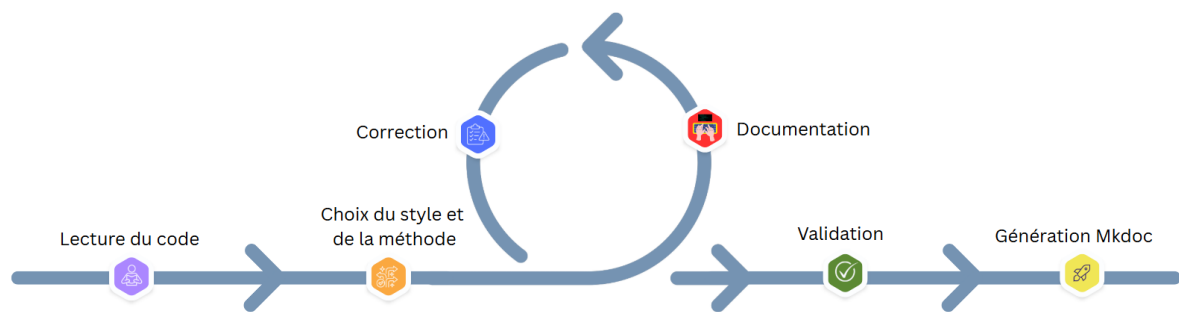


FIGURE 2.1 : Processus de documentation du code source

Afin de rendre cette documentation plus accessible, nous avons également produit un site statique avec *MkDocs* et *mkdocstrings*. Cet outil permet de générer automatiquement des pages de documentation à partir des informations présentes dans le code. Le résultat est plus simple à consulter qu'une lecture directe des fichiers Python.

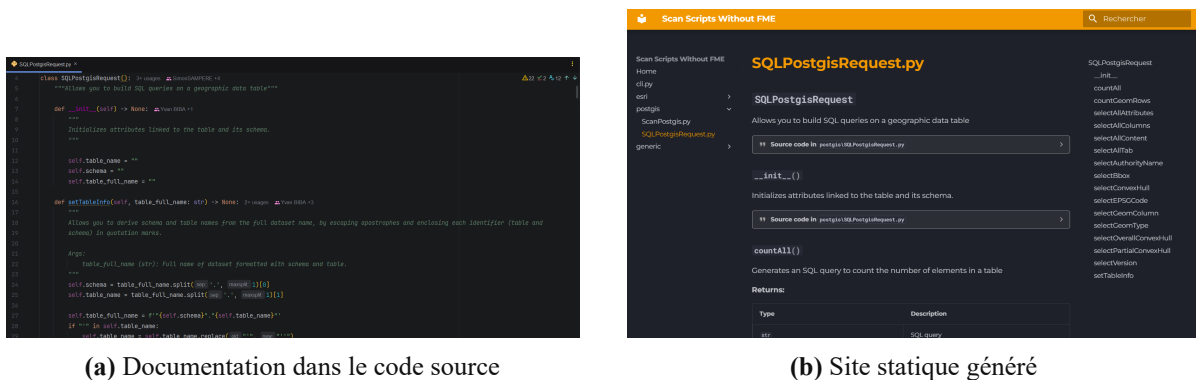


FIGURE 2.2 : Passage du code brut au site statique de documentation

2.3 Expérimentations avec GDAL et OGR

2.3.1 Prise en main des bibliothèques

GDAL et OGR sont des bibliothèques largement utilisées pour lire, manipuler et analyser des données géographiques. Dans le cadre de la mission, elles représentaient une solution importante pour remplacer une partie des traitements historiquement réalisés avec FME.

La prise en main a d'abord porté sur la lecture de fichiers vectoriels et raster. Il s'agissait de vérifier comment accéder aux couches, aux champs, au système de coordonnées, à l'emprise ou encore au nombre d'entités.

Ces premiers essais ont permis de mieux comprendre la manière dont GDAL et OGR présentent les données. Ils ont aussi montré que les informations disponibles peuvent varier selon les formats, les pilotes utilisés et la qualité des données en entrée.

2.3.2 Réalisation de tests techniques

Des tests techniques ont ensuite été réalisés sur les formats rasters et vecteurs. L'objectif était de vérifier ce que les bibliothèques permettaient d'obtenir et de repérer les comportements particuliers dès le début du travail.

Ces tests ont porté sur des opérations simples, comme l'ouverture d'une source, la lecture des couches, la récupération des champs ou l'accès aux informations géographiques principales. Ils ont aussi permis d'observer les erreurs renvoyées lorsque la donnée était absente, invalide ou mal structurée.

Cette étape a été utile pour préparer la suite du développement. Elle a permis d'identifier les possibilités offertes par GDAL et OGR, mais aussi certaines limites à prendre en compte dans l'architecture de la solution.

2.4 Conception de la solution

2.4.1 Définition de l'architecture

Après la phase de prise en main, il a été nécessaire de définir une organisation claire pour les nouveaux traitements. L'objectif était de construire une solution capable de prendre en charge plusieurs sources, sans dupliquer inutilement la logique commune.

L'architecture devait séparer les responsabilités. Les éléments communs devaient gérer les paramètres d'entrée, la structure des résultats, les erreurs et les étapes générales du traitement. Les éléments propres à chaque source devaient se concentrer sur les particularités du format ou de la technologie utilisée.

Cette organisation permettait de faciliter les évolutions futures. Lorsqu'une nouvelle source devait être prise en charge, l'idée était de pouvoir réutiliser le socle commun et d'ajouter uniquement les traitements spécifiques nécessaires.

La conception devait aussi tenir compte de l'intégration dans Scan. Les scripts ne devaient pas seulement fonctionner seuls. Ils devaient produire des résultats exploitables par le reste du produit et respecter les conventions déjà en place.

2.4.2 Définition de la méthode de validation

La méthode de validation dépendait du type de traitement étudié. Pour l'inventaire des données et la documentation, elle reposait principalement sur des tests de non-régression. L'objectif était de comparer les sorties produites par les nouveaux scripts Python avec les résultats obtenus par les workbenches FME de référence.

Pour la documentation des données, la comparaison portait sur l'ensemble des métadonnées

produites : nom de la ressource, système de projection, emprise, enveloppe convexe, champs, types de géométrie et autres informations attendues par Scan.

Pour la signature, la validation cherchait surtout à vérifier l'intégrité du calcul. Il fallait s'assurer que le hash restait stable lorsque la donnée ne changeait pas, mais qu'il évoluait lorsqu'une modification réelle était effectuée.

Type de test	Objectif	Application
Non-régression	Comparer les sorties Python avec les résultats FME de référence.	Inventaire et documentation.
Intégrité de la signature	Vérifier que le hash réagit correctement aux modifications de la donnée.	Signature.
Performance	Mesurer le comportement des scripts sur des volumes de données importants.	Inventaire, documentation et signature.

TABLE 2.1 : Principaux tests prévus pour valider les traitements Python

NB : les tests d'intégrité de la signature regroupaient plusieurs vérifications. La **constance** permettait de vérifier qu'une même donnée produisait la même signature entre deux exécutions. Les **cas limites** servaient à contrôler le comportement face à une table absente, une source invalide ou des types inconnus. L'**isolation** vérifiait qu'une modification sur une couche ne modifiait pas la signature d'une autre couche. Les tests **spatiaux** portaient sur les modifications géométriques ou d'emprise. Les tests **structurels** permettaient de détecter un changement de schéma, comme l'ajout d'une colonne. Enfin, les tests de **sensibilité** vérifiaient que toute modification significative de la donnée était bien prise en compte dans le hash.

Ces tests permettaient d'identifier rapidement les écarts avec le comportement historique de Scan. Lorsqu'une différence était détectée, elle était analysée pour déterminer s'il s'agissait d'une régression, d'un cas particulier ou d'un ajustement acceptable.

2.5 Préparation du packaging

2.5.1 Objectifs du packaging

Un autre enjeu important concernait la distribution des traitements Python. La solution devait pouvoir être exécutée dans un environnement Windows sans installation locale de Python.

Le packaging avait donc pour objectif de regrouper les scripts, les bibliothèques nécessaires et les dépendances géospatiales dans un exécutable autonome. Cette approche permettait de

simplifier le déploiement et de limiter les prérequis côté utilisateur.

La difficulté principale venait des bibliothèques géospatiales. GDAL et OGR sont des outils puissants, mais leur installation peut être complexe dans un environnement Windows classique. Ils reposent sur plusieurs dépendances natives et leur configuration peut varier selon les versions installées.

Il était donc important de produire un exécutable qui embarque directement ces éléments. L'objectif était d'éviter aux utilisateurs et aux développeurs de se poser des questions sur l'installation manuelle des modules, en particulier pour GDAL. Le processus de génération devait donc être fiable, reproductible et maîtrisé.

2.5.2 Mise en place d'un environnement Docker

Pour répondre à ce besoin, la génération de l'exécutable devait être réalisée dans un environnement Docker. L'objectif était d'avoir Docker comme seul prérequis pour construire l'exécutable.

Cette organisation permettait de maîtriser l'environnement de construction. Les versions de Python, de PyInstaller, de GDAL et des autres dépendances pouvaient être fixées dans une image dédiée. La génération ne dépendait donc plus des bibliothèques installées sur le poste du développeur.

Plusieurs approches ont été étudiées pour intégrer GDAL dans cet environnement. Les premiers essais ont notamment porté sur les wheels Python, c'est-à-dire des paquets déjà préparés pour faciliter l'installation des bibliothèques. Cette solution était simple à tester, mais la source utilisée pour ces wheels n'était pas suffisamment stable ni maintenue de façon durable. Pour une solution comme Isogeo, qui doit rester fiable dans le temps, cette approche n'était donc pas la plus adaptée.

Une autre piste consistait à utiliser OSGeo, une distribution Windows regroupant plusieurs outils de géomatique, dont GDAL. Cette approche avait l'avantage d'être proche des installations classiques du domaine. En revanche, elle produisait un livrable beaucoup plus lourd et dépendait de chemins d'installation susceptibles de changer selon les versions. Cela risquait de rendre le Dockerfile plus difficile à maintenir.

L'approche Conda a finalement été retenue. Conda permet de créer un environnement isolé avec des versions précises des outils utilisés. Cela permettait de figer plus facilement les versions de Python, GDAL, PyInstaller et des bibliothèques associées. Cette solution produisait également un dossier final plus léger et un environnement plus stable.

Le processus a donc été organisé en deux étapes. Une première image Docker servait de base et contenait l'environnement Python avec les dépendances géospatiales principales. Une seconde image, dédiée à la construction, ajoutait le code source de Scan, installait les dépendances propres au projet et lançait le script de génération de l'exécutable.

Cette séparation facilitait la maintenance. L'image de base pouvait être réutilisée tant que les

versions principales ne changeaient pas. L'image de construction pouvait ensuite être régénérée à partir du code courant.

Un point particulier concernait les données internes utilisées par GDAL, notamment celles de la bibliothèque PROJ. Elles devaient être correctement embarquées avec l'exécutable afin que l'application utilise les bonnes données au moment de son exécution, sans dépendre d'une installation présente sur la machine.

Cette étape préparait l'industrialisation de la solution. Elle devait permettre de passer progressivement de scripts de développement à un programme Windows généré de manière fiable, reproductible et utilisable dans le contexte réel de Scan.

CHAPITRE 3

Développement des scripts Python

3.1 Organisation générale des développements

3.1.1 Une progression par formats de données

Le développement des nouveaux traitements Python ne s'est pas fait en une seule fois. Il a été réalisé progressivement, format par format en fonction des priorités du produit et des difficultés rencontrées.

Cette organisation permettait de travailler par étapes. Pour chaque format, il fallait comprendre les données disponibles, identifier les métadonnées utiles, développer le traitement adapté, puis vérifier les résultats obtenus.

Le travail a d'abord porté sur des formats qui étaient les plus utilisés par les clients de l'entreprise, comme les fichiers rasters et vectoriels. Il s'est ensuite poursuivi avec des formats ou technologies plus spécifiques : GeoPackage, données CAO, Oracle, GeoParquet, SQL Server et, plus récemment, Nuage des points.

Cette progression correspondait aussi à la manière dont la migration devait être menée dans Scan. Chaque ajout devait rester compatible avec le fonctionnement existant du produit. Il ne s'agissait donc pas seulement de lire une donnée, mais de produire une sortie exploitable par les autres composants.

3.1.2 Périmètre fonctionnel selon les formats

Le périmètre n'était pas identique pour tous les formats. Certains ont fait l'objet d'un travail de documentation des métadonnées. D'autres ont nécessité des traitements complémentaires, comme le listing ou la signature.

Pour les fichiers rasters, vectoriels et CAO, le travail réalisé concernait uniquement la documentation. L'objectif était d'extraire les informations techniques nécessaires à la création ou à la mise à jour des fiches de métadonnées dans Scan. Le listing et la signature n'ont pas été développés pour ces formats, car ils pouvaient être réalisés directement par le client de Scan.

Pour les formats suivants, le périmètre dépendait des besoins du produit, de la nature des données et des possibilités offertes par les bibliothèques utilisées. Cette approche progressive permettait d'avancer sans figer trop tôt une solution unique qui n'aurait pas forcément convenu à toutes les technologies.

Format ou technologie	Travail réalisé ou en cours
Rasters, vecteurs et CAO	Documentation.
GeoPackage	Inventaire, documentation et signature.
Oracle	Inventaire, documentation et signature.
GeoParquet	Documentation.
SQL Server	Inventaire, documentation et signature.
Nuage des points	Travaux en cours sur l'inventaire, la documentation et la signature.

TABLE 3.1 : Progression des développements selon les formats étudiés

3.2 Développements sur les fichiers géographiques

3.2.1 Données rasters et vectorielles

Les premiers développements ont porté sur les données rasters et vectorielles. Ces formats constituaient un bon point d'entrée, car ils permettaient de travailler sur des formats fréquents dans les usages géomatiques.

Pour les données vectorielles, deux formats ont été pris en compte dans un premier temps : le Shapefile et le GeoJSON. Ces formats sont très utilisés pour échanger des couches géographiques. Le traitement devait donc être capable de lire la couche, d'identifier sa géométrie, de récupérer les champs attributaires et de produire les métadonnées attendues par Scan.

Pour les données rasters, plusieurs formats ont été pris en charge, notamment ECW, GeoTIFF, JP2, JPG, PNG et d'autres formats courants. Les informations à extraire n'étaient pas exactement les mêmes que pour les données vectorielles. Il fallait par exemple tenir compte de la taille de l'image, du nombre de bandes, du système de coordonnées et de l'emprise.

Pour cette première partie, seule la documentation (lookup) a été développée. Le but était de produire une description fiable de la ressource analysée, sans développer le listing ni la signature, qui pouvaient déjà être réalisés par le client de Scan pour ces formats.

GDAL et OGR ont été utilisés pour accéder à ces informations. Ces bibliothèques offrent une interface commune pour lire de nombreux formats. Elles permettent donc de limiter le nombre de traitements spécifiques à écrire pour chaque type de fichier.

Ce premier travail a permis de mettre en place une base de développement. Il a aussi permis d'identifier les premières différences de comportement entre les formats, notamment sur la manière dont les systèmes de coordonnées, les champs ou l'enveloppe convexe sont exposés.

3.2.2 GeoPackage

Le travail s'est ensuite poursuivi avec le format GeoPackage. Il s'agit d'un format de fichier permettant de stocker plusieurs couches géographiques dans un même fichier. Il est souvent utilisé pour échanger des données tout en conservant une structure proche d'une petite base de données.

La prise en charge de GeoPackage nécessitait donc de parcourir les couches présentes dans le fichier, puis de produire les informations attendues pour chacune d'elles. Le traitement devait rester cohérent avec la logique de Scan : une ressource identifiée devait pouvoir être documentée de manière claire et stable.

Le développement a porté sur les trois traitements attendus : l'inventaire, la documentation et la signature. L'inventaire permettait de lister les couches présentes dans le fichier. La documentation produisait les métadonnées de chaque couche. La signature permettait ensuite de détecter les changements entre deux analyses.

Ce format demandait une attention particulière, car un seul fichier peut contenir plusieurs couches. Il fallait donc bien distinguer la ressource globale du fichier et les couches réellement exploitées par Scan. Les résultats ont été vérifiés sur plusieurs jeux de données afin de contrôler la cohérence des sorties.

GeoPackage a aussi permis de confirmer l'importance d'une architecture souple. Même si le format est stocké dans un fichier, son fonctionnement se rapproche sur certains points d'une base de données. Le code devait donc éviter de traiter tous les fichiers de manière trop uniforme.

3.2.3 GeoParquet

GeoParquet a ensuite été étudié comme format de stockage géographique plus récent. Il s'appuie sur le format Parquet, souvent utilisé pour stocker des données de manière efficace, notamment dans des contextes d'analyse.

L'objectif était d'identifier les informations disponibles dans ce format et de voir comment les exploiter pour produire les métadonnées attendues par Scan. Le traitement devait notamment retrouver les informations géographiques, les champs et les éléments nécessaires à la description de la ressource.

Pour ce format, le travail a porté sur la documentation. Il fallait comprendre comment les informations spatiales étaient stockées dans le fichier et comment retrouver les éléments utiles : champs, géométrie, système de coordonnées et emprise. Ce format a nécessité une phase d'expérimentation, car il ne se présente pas de la même manière qu'un fichier SIG traditionnel.

Les informations utiles ne sont pas toujours exposées avec les mêmes conventions que dans d'autres formats. Le traitement devait donc rester prudent dans la lecture des métadonnées et dans l'interprétation des informations géographiques.

La prise en charge de GeoParquet a donc permis d'élargir le périmètre de la migration vers des formats plus modernes, tout en conservant la même exigence : produire des résultats exploitables et comparables dans Scan.

3.2.4 Données CAO

Une autre étape a concerné les données CAO. Ces données proviennent du dessin assisté par ordinateur et ne suivent pas toujours les mêmes logiques que les données SIG classiques.

Le travail sur ces formats a d'abord été engagé avec GDAL et OGR, comme pour les premiers formats étudiés. Cette piste semblait cohérente, car ces bibliothèques permettaient déjà de lire plusieurs types de données géographiques.

Cependant, cette approche a rapidement montré ses limites. Le format DWG, important pour les données CAO, n'était pas pris en charge par GDAL et OGR. Il a donc fallu rechercher une autre solution pour pouvoir exploiter ces données dans le cadre de Scan.

Après cette phase d'étude, la solution retenue a été *LibreDWG*. Elle a permis de traiter les données CAO et d'en extraire les informations réellement exploitables pour la documentation. Ces formats peuvent contenir des structures particulières, des géométries différentes ou des informations moins directement normalisées.

Certaines limitations dépendaient des technologies utilisées. Les formats SDF et DGN ont notamment dû être mis de côté, car leur prise en charge n'était pas suffisamment fiable dans le contexte de la migration.

Pour les données CAO, le travail est donc resté centré sur la documentation. L'objectif était d'extraire les informations réellement disponibles et utiles à Scan, sans chercher à forcer une interprétation SIG complète lorsque le format ne s'y prêtait pas. Cette étape a montré qu'une migration ne consiste pas toujours à tout reprendre immédiatement. Dans certains cas, il est préférable d'identifier clairement les limites, de documenter les raisons du blocage et de concentrer les efforts sur les formats pouvant être traités de manière fiable.

3.3 Développements sur les bases de données

Pour les bases de données, le choix a été différent de celui retenu pour les formats fichiers. Nous avons privilégié l'utilisation de bibliothèques natives, basées sur l'exécution de requêtes SQL. Cette approche permettait de s'appuyer sur les capacités propres des bases de données et de bénéficier de leurs performances, notamment lorsque les volumes à traiter étaient importants.

3.3.1 Oracle

Après les premiers travaux sur les fichiers, le développement s’est poursuivi sur Oracle. Cette technologie introduisait des contraintes différentes, car les données sont stockées dans une base relationnelle, avec des paramètres de connexion, des schémas, des tables et des droits d’accès.

Le traitement devait donc gérer la connexion à la base, identifier les tables accessibles et récupérer les informations utiles à la documentation. Il fallait aussi tenir compte des particularités d’Oracle Spatial pour les données géographiques.

Pour Oracle, la bibliothèque *python-oracledb* a été utilisée. Elle permettait d’exécuter directement les requêtes nécessaires pour l’inventaire, la documentation et la signature. Cette solution évitait de rapatrier inutilement les données côté Python lorsque la base pouvait réaliser elle-même certaines opérations.

Cette étape a demandé une attention particulière sur la robustesse. Une base de données peut être inaccessible, une table peut ne pas contenir de géométrie ou un utilisateur peut ne pas avoir les droits nécessaires. Le traitement devait donc produire un comportement clair dans ces situations.

Le travail sur Oracle a renforcé l’idée que chaque format ou technologie possède ses propres contraintes. Même lorsque les métadonnées recherchées sont proches, la manière de les obtenir peut changer fortement selon la technologie.

3.3.2 SQL Server

Le développement a également concerné Microsoft SQL Server. Comme pour Oracle, il s’agit d’une base relationnelle pouvant contenir des données spatiales.

La prise en charge de cette technologie impliquait de gérer les paramètres de connexion, les bases, les schémas et les tables accessibles. Il fallait aussi tenir compte des types géographiques disponibles dans SQL Server et de la façon dont les informations spatiales peuvent être interrogées.

Pour SQL Server, le choix s’est porté sur *SQLAlchemy*. Cette bibliothèque permettait de structurer les accès à la base tout en conservant une logique reposant sur des requêtes SQL. Comme pour Oracle, l’objectif était de profiter des capacités de la base pour limiter les traitements inutiles côté application.

Le travail a consisté à adapter les traitements aux spécificités de cet environnement, tout en conservant une sortie cohérente avec les autres formats et technologies. L’objectif restait le même : permettre à Scan de recenser ou documenter les ressources sans dépendre des anciens traitements.

Cette étape a aussi permis de renforcer la gestion des erreurs et des cas particuliers. Les bases de données apportent souvent plus de variabilité que les fichiers, car leur contenu dépend de

l'organisation mise en place chez chaque client.

3.4 Traitement des nuages de points

Les travaux les plus récents portent sur le Nuage des points. Ce type de donnée est particulier, car il peut représenter des volumes très importants et ne se manipule pas comme une couche vectorielle classique.

L'objectif est d'identifier les métadonnées pertinentes pour décrire ces ressources dans Scan. Il faut notamment comprendre les informations disponibles, les formats utilisés et les limites liées au volume des données.

Cette partie est encore en cours. Elle nécessite de prendre en compte les contraintes de performance, car les fichiers peuvent être très lourds. Le traitement doit donc éviter les opérations trop coûteuses lorsque seules certaines métadonnées sont nécessaires.

Le Nuage des points illustre bien la logique progressive de la migration. Chaque nouveau format apporte ses propres contraintes et oblige à adapter les traitements sans perdre la cohérence générale de la solution.

3.5 Packaging et exécutable au fil des développements

3.5.1 Réutilisation de l'environnement Docker

La génération de l'exécutable n'a pas été traitée comme une étape finale indépendante. Une fois l'image Docker de construction mise en place, elle a été réutilisée au fur et à mesure de l'avancement des scripts.

Cette organisation permettait de vérifier régulièrement que les développements pouvaient toujours être intégrés dans un exécutable Windows. Elle évitait de découvrir trop tard un problème lié au packaging ou à une dépendance.

Docker restait le seul prérequis pour générer l'exécutable. L'environnement de construction contenait les versions nécessaires de Python, PyInstaller, GDAL et des autres bibliothèques utilisées par Scan.

3.5.2 Vérification de l'exécutable

Après l'ajout ou la modification de certains traitements, l'exécutable pouvait être régénéré afin de vérifier son bon fonctionnement. Les tests portaient alors sur l'exécution du programme, la présence des dépendances nécessaires et la cohérence des résultats produits.

Une attention particulière était portée aux bibliothèques géospatiales, car elles reposent sur des dépendances natives. Le packaging devait donc embarquer les éléments nécessaires pour éviter de dépendre d'une installation locale sur la machine utilisée.

Cette vérification régulière permettait de garder un lien direct entre les scripts développés et leur utilisation réelle dans Scan. Elle contribuait aussi à sécuriser progressivement la migration vers une solution Python autonome.